

编 号	题 号	标 题	知 识 点	代 码 作 者
例 7-9	UVa 247	Calling Circles	Floyd 算法：传递闭包	陈锋
例 7-10	UVa 10048	Audiophobia	Floyd 算法：最大值最小路	陈锋
例 7-11	UVa 11374	Airport Express	用 Dijkstra 预处理：枚举	刘汝佳
例 7-12	UVa 1416	Warfare And Logistics	最短路树	刘汝佳
例 7-13	UVa 10537	Toll! Revisited	Dijkstra 算法的变形	刘汝佳
例 7-14	UVa 658	It's not a Bug	复杂状态的最短路	陈锋
例 7-15	UVa 1078	Steam Roller	状态图设计：Dijkstra	刘汝佳
例 7-16	UVa 12661	Funny Car Racing	特殊图的 Dijkstra 算法	刘汝佳
例 7-17	UVa 11090	Going in Cycle!!	01 分数规划：负圈判定	刘汝佳
例 7-18	UVa 11478	Halum	差分约束系统	刘汝佳

7.3 生成树相关问题

本节介绍与最小生成树以及最小有向生成树算法有关的几个经典例题以及代码实现。

例7-19 【最小生成树】苗条的生成树 (Slim Span, ACM/ICPC Japan 2007, POJ 3522)

给出一个 n ($n \leq 100$) 个结点的图，求苗条度（最大边减去最小边的值）尽量小的生成树。

【代码实现】

```

// 陈锋
#include <algorithm>
#include <cstdio>
#include <iostream>
#include <vector>
using namespace std;

const int maxn = 100 + 10, INF = 1e9;
int N, pa[maxn]; // 并查集相关代码
int find_pa(int x) { return pa[x] != x ? pa[x] = find_pa(pa[x]) : x; }

struct Edge {
    int u, v, d;
    Edge(int u, int v, int d) : u(u), v(v), d(d) {}
    bool operator<(const Edge& rhs) const { return d < rhs.d; }
};

vector<Edge> E;
int solve() { // Kruscal 算法
    int m = E.size(), ans = INF;
    sort(E.begin(), E.end());
    for (int L = 0; L < m; L++) {
        for (int i = 1; i <= N; i++) pa[i] = i;
        for (int R = L, cnt = N; R < m; R++) {
            int u = find_pa(E[R].u), v = find_pa(E[R].v);
            if (u == v) continue;
            pa[u] = v;
            if (--cnt == 1) { // 连通分量只剩一个
                ans = min(ans, E[R].d - E[L].d);
                break;
            }
        }
    }
    if (ans == INF) ans = -1;
    return ans;
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    for (int m; cin >> N >> m && N; E.clear()) {
        for (int i = 0, u, v, d; i < m; i++)
            cin >> u >> v >> d, E.push_back(Edge(u, v, d));
        printf("%d\n", solve());
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典(第2版)》例题 11-2
同类题目: Qin Shi Huang's National Road System, ACM/ICPC Beijing 2011, HDU 4081
*/

```

例7-20 【最小生成树；动态LCA】邦德 (Bond, UVa 11354)

有 n 座城市通过 m ($2 \leq n \leq 50\,000$, $1 \leq m \leq 100\,000$) 条双向道路相连，每条道路都有一个危险系数。你的任务是回答 Q ($1 \leq Q \leq 50\,000$) 个询问，每个询问包含一个起点 s 和一个终点 t ，要求找到一条从 s 到 t 的路，使得途经的所有边的最大危险系数最小。输出最优路线上所有边的危险系数的最大值。

【代码实现】

```
// 陈锋
#include <bits/stdc++.h>

using namespace std;
#define _for(i, a, b) for (int i = (a); i < (int)(b); ++i)
const int MAXN = 50000 + 4;
struct Edge {
    int u, v, w;
    Edge(int _u = 0, int _v = 0, int _w = 0) : u(_u), v(_v), w(_w) {}
    bool operator<(const Edge& e) const {
        return w < e.w;
    }
};

int N, M;
vector<Edge> G[MAXN]; // MST 多生成树
int L, Tin[MAXN], Tout[MAXN], UP[MAXN][20], MaxW[MAXN][20], timer;

bool isAncestor(int u, int v) { return Tin[u] <= Tin[v] && Tout[u] >= Tout[v]; }

void dfs(int u, int fa, int w) {
    Tin[u] = ++timer, UP[u][0] = fa, MaxW[u][0] = w;
    for (int i = 1; i <= L; i++) {
        int ui = UP[u][i - 1];
        UP[u][i] = UP[ui][i - 1];
        MaxW[u][i] = max(MaxW[u][i - 1], MaxW[ui][i - 1]);
    }
    _for(i, 0, G[u].size()) {
```

```

        const Edge& e = G[u][i];
        if (e.v != fa) dfs(e.v, u, e.w);
    }
    Tout[u] = ++timer;
}

int LCA(int u, int v) {
    if (isAncestor(u, v)) return u;
    if (isAncestor(v, u)) return v;
    for (int i = L; i >= 0; --i) if (!isAncestor(UP[u][i], v)) u = UP[u][i];
    return UP[u][0];
}

int find_maxw(int u, int v) { // v → u 路径上的最大 w, u = Ancestor(v)
    if (u == v) return 0;
    assert(isAncestor(u, v));
    int w = 0;
    for (int i = L; i >= 0; --i) // 保证 u 是 v 的祖先, 且 u != v
        if (!isAncestor(UP[v][i], u) && UP[v][i] != u)
            w = max(w, MaxW[v][i]), v = UP[v][i];
    assert(UP[v][0] == u);
    return max(w, MaxW[v][0]);
}

int PA[MAXN]; // 并查集
int find_pa(int i) { return PA[i] == i ? i : (PA[i] = find_pa(PA[i])); }

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    vector<Edge> es;
    for (int kase = 0, Q; cin >> N >> M; kase++) {
        L = ceil(log2(N));
        if (kase) puts("");
        es.clear();
        Edge e;
        _for(i, 0, M) cin >> e.u >> e.v >> e.w, es.push_back(e);
        sort(es.begin(), es.end());
        for (int i = 0; i <= N; i++) PA[i] = i, G[i].clear();
        _for(i, 0, es.size()) {
            const Edge& e = es[i];
            int u = e.u, v = e.v, pu = find_pa(u), pv = find_pa(v);
            if (pu != pv) {
                PA[pv] = pu;
                G[u].push_back(Edge(u, v, e.w)), G[v].push_back(Edge(v, u, e.w));
            }
        }
        timer = 0, dfs(1, 1, 0); // MST 上计算 LCA 以及相关的倍增结构
        cin >> Q;
        for (int i = 0, s, t; i < Q; i++) {

```

```

    cin >> s >> t;
    int l = LCA(s, t);
    assert(s != t);
    printf("%d\n", max(find_maxw(l, s), find_maxw(l, t)));
}
}
return 0;
}

```

/*

算法分析请参考：《算法竞赛入门经典——训练指南》升级版 5.4 节例题 21

注意：本题也使用了基于时间戳的倍增 LCA

同类题目：Imperial roads, ACM/ICPC 2017 Latin American, Codeforces Gym 101889I

*/

例7-21 【最小有向生成树】比赛网络 (Stream My Contest, UVa 11865)

你需要花费不超过 cost 元来搭建一个比赛网络。网络中有 n 台机器，编号为 $0 \sim n-1$ ($1 \leq n \leq 60$, $1 \leq \text{cost} \leq 10^9$)，其中机器0为服务器，其他机器为客户机。一共有 m ($1 \leq m \leq 10\,000$)条可以使用的网线，其中第 i 条网线的发送端是机器 u_i ，接受端是机器 v_i （数据只能从机器 u_i 单向传输到机器 v_i ），带宽是 b_i (Kbps)，费用是 c_i 元 ($0 \leq u_i, v_i < n$, $1 \leq b_i, c_i \leq 10^6$, $u_i \neq v_i$)。每台客户机应当恰好从一台机器接收数据（即恰好有一条网线的接收端是该机器），而服务器不应从任何机器接收数据。你的任务是最大化网络中的最小带宽。

输出最小带宽的最大值。如果无法搭建网络，输出“streaming not possible.”（不含引号）。

【代码实现】

```

// 刘汝佳
#include <bits/stdc++.h>
using namespace std;
const int INF = 1e9, maxn = 100 + 10;

// 固定根的最小树型图，邻接矩阵写法
struct MDST {
    int n;
    int w[maxn][maxn];           // 边权
    int vis[maxn];               // 访问标记，仅用来判断无解
    int ans;                     // 计算答案
    int removed[maxn];          // 每个点是否被删除
    int cid[maxn];               // 所在圈编号
    int pre[maxn];               // 最小入边的起点
    int iw[maxn];                // 最小入边的权值
    int max_cid;                 // 最大圈编号

    void init(int n) {
        this->n = n;
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++) w[i][j] = INF;
    }
};

```

```

void AddEdge(int u, int v, int cost) {
    w[u][v] = min(w[u][v], cost); // 重边取权最小的
}

// 从s出发能到达多少个结点
int dfs(int s) {
    vis[s] = 1;
    int ans = 1;
    for (int i = 0; i < n; i++)
        if (!vis[i] && w[s][i] < INF) ans += dfs(i);
    return ans;
}

// 从u出发沿着pre指针找圈
bool cycle(int u) {
    max_cid++;
    int v = u;
    while (cid[v] != max_cid) { cid[v] = max_cid; v = pre[v]; }
    return v == u;
}

// 计算u的最小入弧，入弧起点不得在圈c中
void update(int u) {
    iw[u] = INF;
    for (int i = 0; i < n; i++)
        if (!removed[i] && w[i][u] < iw[u]) {
            iw[u] = w[i][u];
            pre[u] = i;
        }
}

// 根结点为s，如果失败则返回 false
bool solve(int s) {
    memset(vis, 0, sizeof(vis));
    if (dfs(s) != n) return false;

    memset(removed, 0, sizeof(removed));
    memset(cid, 0, sizeof(cid));
    for (int u = 0; u < n; u++) update(u);
    pre[s] = s; iw[s] = 0; // 根结点特殊处理
    ans = max_cid = 0;
    for (;;) {
        bool have_cycle = false;
        for (int u = 0; u < n; u++) if (u != s && !removed[u] && cycle(u)) {
            have_cycle = true;
            // 以下代码缩圈，圈上除u外的结点均删除
            int v = u;
            do {
                if (v != u) removed[v] = 1;
            } while (v = pre[v]);
        }
        if (!have_cycle) break;
    }
}

```

```

    ans += iw[v];
    // 对于圈外点 i, 把边 i->v 改成 i->u (并调整权值), v->i 改为 u->i
    // 注意, 圈上可能还有一个 v' 使得 i->v' 或 v'->i 存在, 因此只保留权值最小的 i->u
和 u->i
    for (int i = 0; i < n; i++) if (cid[i] != cid[u] && !removed[i]) {
        if (w[i][v] < INF) w[i][u] = min(w[i][u], w[i][v] - iw[v]);
        w[u][i] = min(w[u][i], w[v][i]);
        if (pre[i] == v) pre[i] = u;
    }
    v = pre[v];
} while (v != u);
update(u);
break;
}
if (!have_cycle) break;
}
for (int i = 0; i < n; i++)
    if (!removed[i]) ans += iw[i];
return true;
}
};

///////// 题目相关
MDST solver;
struct Edge {
    int u, v, b, c;
    bool operator < (const Edge& rhs) const {
        return b > rhs.b;
    }
};

const int maxm = 10000 + 10;
int N, M, C;
Edge edges[maxm];

// 取 b 前 cnt 大的边构造网络, 判断最小树型图的边权和是否小于 C
bool check(int cnt) {
    solver.init(N);
    for (int i = 0; i < cnt; i++)
        solver.AddEdge(edges[i].u, edges[i].v, edges[i].c);
    if (!solver.solve(0)) return false;
    return solver.ans <= C;
}

int main() {
    int T;
    scanf("%d", &T);
    while (T--) {
        scanf("%d%d%d", &N, &M, &C);

```



```

for (int i = 0; i < M; i++) {
    scanf("%d%d%d%d", &edges[i].u, &edges[i].v, &edges[i].b, &edges[i].c);
}
sort(edges, edges + M);
int l = 1, r = M, ans = -1;
while (l <= r) {
    int m = l + (r - l) / 2;
    if (check(m)) ans = edges[m - 1].b, r = m - 1;
    else l = m + 1;
}
if (ans < 0) printf("streaming not possible.\n");
else printf("%d kbps\n", ans);
}
return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 5.5 节例题 22
同类题目: Command Network, POJ 3164
*/

```

例7-22 【动点的最小生成树】星际游击队 (Asteroid Rangers, ACM/ICPC WF2012, UVa 1279)

三维空间里有 n ($2 \leq n \leq 50$) 个匀速运动的点, 第 i 个点的初始坐标为 (x, y, z) , 速度为 (v_x, v_y, v_z) 。求最小生成树会改变多少次。输入保证在任意时刻最小生成树总是唯一的, 并且每次变化时, 新的最小生成树至少会保持 10^{-6} 个单位的时间。

【代码实现】

```

// 刘汝佳
#include <algorithm>
#include <cmath>
#include <cstdio>
#include <vector>
using namespace std;

const int maxn = 50 + 5, maxks = maxn * (maxn + 1) / 2;
const double eps = 1e-8;

struct Event {
    double t;
    int newks, oldks;           // 每个事件后, newks 比 oldks 小
    Event(double t = 0, int newks = 0, int oldks = 0)
        : t(t), newks(newks), oldks(oldks) {}
    bool operator<(const Event& rhs) const { return t - rhs.t < 0; }
};

int n, nks;
vector<Event> ES;

struct KineticPoint {

```

```

double x, y, z; // 初始位置
double dx, dy, dz; // 初始速度
void read() { scanf("%lf%lf%lf%lf%lf%lf", &x, &y, &z, &dx, &dy, &dz); }
} kp[maxn];

struct KineticSegment {
    double a, b, c; // 长度是  $at^2+bt+c$ 
    int u, v; // 端点
    bool operator<(const KineticSegment& rhs) const { // 初始长度比较
        return c - rhs.c < 0;
    }
} ks[maxks];

inline double sqr(double x) { return x * x; }

int pa[maxn];

void init_ufset() {
    for (int i = 0; i < n; i++) pa[i] = i;
}
int findset(int x) { return pa[x] != x ? pa[x] = findset(pa[x]) : x; }

void make_segments() {
    nks = 0;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++) {
            /*
            点 i 和点 j 的距离平方是  $\text{sum}((kp[i].dx-kp[j].dx)^2 + (kp[i].x-kp[j].x)^2)$ 
            可以写成  $at^2 + bt + c$ .  $a > 0, c > 0$ 
            */
            ks[nks].a = sqr(kp[i].dx - kp[j].dx) + sqr(kp[i].dy - kp[j].dy) +
                sqr(kp[i].dz - kp[j].dz);
            ks[nks].b = 2 * ((kp[i].dx - kp[j].dx) * (kp[i].x - kp[j].x) +
                (kp[i].dy - kp[j].dy) * (kp[i].y - kp[j].y) +
                (kp[i].dz - kp[j].dz) * (kp[i].z - kp[j].z));
            ks[nks].c = sqr(kp[i].x - kp[j].x) + sqr(kp[i].y - kp[j].y) +
                sqr(kp[i].z - kp[j].z);
            ks[nks].u = i;
            ks[nks].v = j;
            nks++;
        }
    sort(ks, ks + nks);
}

void make_events() {
    ES.clear();
    for (int i = 0; i < nks; i++)
        for (int j = i + 1; j < nks; j++) {
            int s1 = i, s2 = j; // 何时线段 i 和 j 长度相等

```

```

    if (ks[s1].a - ks[s2].a < 0)
        s1 = j, s2 = i; // s1 更陡

    double a = ks[s1].a - ks[s2].a;
    double b = ks[s1].b - ks[s2].b;
    double c = ks[s1].c - ks[s2].c;
    if (fabs(a) < eps) { // bt + c = 0
        if (fabs(b) < eps) continue; // 无解
        if (b > 0) {
            swap(s1, s2);
            b = -b;
            c = -c;
        } // bt + c = 0, b < 0
        if (c > 0) ES.push_back(Event(-c / b, s1, s2)); // t > 0
        continue;
    }
    double delta = b * b - 4 * a * c;
    if (delta < eps) continue; // 无解
    delta = sqrt(delta);
    double t1 = -(b + delta) / (2 * a); // 答案 1
    double t2 = (delta - b) / (2 * a); // 答案 2
    if (t1 > 0) ES.push_back(Event(t1, s1, s2)); // 更陡的更小
    if (t2 > 0) ES.push_back(Event(t2, s2, s1)); // 更平的更小
}
sort(ES.begin(), ES.end());
}

int solve() {
    int pos[maxks]; // pos[i]: 线段 i 在 MST 中的编号, 0 表示不在 MST 中
    int e[maxn]; // e[i] 是 MST 中的第 i 条边, pos[e[i]] = i
    init_uisset(); // 计算初始的 MST
    for (int i = 0; i < nks; i++) pos[i] = 0;
    int idx = 0;
    for (int i = 0; i < nks; i++) {
        int u = findset(ks[i].u), v = findset(ks[i].v);
        if (u != v) {
            e[pos[i] = ++idx] = i;
            pa[u] = v;
        }
        if (idx == n - 1) break;
    }

    int ans = 1;
    for (int i = 0; i < ES.size(); i++) {
        if (pos[ES[i].oldks] && (!pos[ES[i].newks])) {
            init_uisset();
            int oldpos = pos[ES[i].oldks];
            for (int j = 1; j < n; j++)
                if (j != oldpos) {

```